

Designed Language Overview

This project introduces a simple programming language named **SpellCraft**, designed specifically for educational purposes in a Compiler Design course. The language follows a fantasy-inspired theme, where variables are referred to as *spells* and keywords are related to magic and alchemy. The main objective of the language is to demonstrate the fundamental concepts of lexical analysis, syntax analysis, symbol table management, scope handling, and semantic analysis using **Flex** and **Bison**.

Data Types

SpellCraft supports two primitive data types:

- **mana**: Represents integer values.
- **elixir**: Represents floating-point values.

Variables can be declared with or without an initial value.

Example:

```
mana $health~  
elixir $damage~
```

```
mana $score => 100~  
elixir $power => 25.5~
```

Identifiers

All identifiers must begin with the \$ symbol, followed by one or more English letters or digits. No other special characters are allowed in variable names.

Identifier format:

```
\$[a-zA-Z0-9]+
```

Examples of valid identifiers:

```
$mana  
$hp1  
$123  
$magic99
```

Operators

The language provides a small set of arithmetic and comparison operators.

Operator	Description
=>	Assignment
+	Addition
-	Subtraction
*	Multiplication
/	Division
==	Equality comparison

The assignment operator is intentionally defined as => to avoid ambiguity with the equality operator (==).

Statement Terminator

Every statement in SpellCraft must end with the ~ symbol. This design simplifies the parsing process by clearly indicating the end of each statement.

Example:

```
mana $x~  
$x => 10~
```

Expressions

SpellCraft supports arithmetic expressions using the four basic arithmetic operators. Parentheses can be used to control operator precedence.

Example:

```
mana $a => 10~  
mana $b => 5~  
mana $c => ($a + $b) * 2~
```

If at least one operand in an expression is of type **elixir**, the resulting expression is also considered to be of type **elixir**. Otherwise, the result is of type **mana**.

Blocks and Scope

A new scope is created using the keywords `begin_spell` and `end_spell`.

Example:

```
begin_spell  
  mana $x~  
end_spell
```

Each block introduces a separate scope. Variables declared inside a block are only accessible within that block and its nested blocks. When execution leaves a block, all variables belonging to that scope are removed from the symbol table.

Redeclaring a variable within the same scope is not allowed. However, declaring a variable with the same name inside a nested scope (shadowing) is permitted.

Conditional Statements

SpellCraft supports conditional execution using the `cast_if` statement with an optional `cast_else` branch.

General syntax:

```
cast_if (condition)
begin_spell
    ...
end_spell
```

```
cast_else
begin_spell
    ...
end_spell
```

Conditions currently support only the equality operator (`==`).

Example:

```
cast_if ($x == $y)
begin_spell
    mana $result => 1~
end_spell
```

Semantic Error Checking

In addition to lexical and syntax analysis, the compiler performs several semantic checks, including:

- Detecting the use of undefined variables.
- Preventing duplicate declarations within the same scope.
- Checking type compatibility during assignments.
- Displaying a warning when assigning an **elixir** value to a **mana** variable.

These semantic checks improve the reliability of the language while keeping the compiler implementation simple and suitable for educational purposes.

Overall, SpellCraft is a lightweight programming language that demonstrates the essential components of a compiler, including lexical analysis, parsing, symbol table management, scope handling, and basic semantic analysis, making it an appropriate project for learning compiler construction techniques.